

# Formal Review Request Template

**Requester:** Devin Veasley  
**Reviewer:** Nishanth Manoj

**Date:** 10/6/2026  
**Approx. review duration:** 1 hour

---

Choose one primary review method:

## Code / Software Review

### Code Quality Review

Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.

### White-Box / Edge-Case Testing

Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.

### Black-Box / Functional Testing

Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.

### Performance / Runtime Testing

Systematically evaluate whether the software meets relevant runtime or performance expectations.

## Usability Review

- **Task-Based Usability Testing**

Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.

- **Heuristic / Guidelines Evaluation**

Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.

- **Prototype Usability Testing**

Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

## Design Review X

- **Architecture / Conceptual Design Review**

Evaluate whether architecture and major design decisions align with project requirements and constraints.

- **Preliminary Design / UML Review**

Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.

- **Security Design Review**

Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

## Other

- **Other review method:**

---

---

**Explain what the reviewer will evaluate and why this method is appropriate for your work:**

The reviewer will be evaluating and grading the architecture, design, and security documentation of the Farmers Well-Being app. This method would be appropriate as myself and team have made it our primary goal to use this sprint to flesh out ideas

## Review Request Details

1) What specifically should the reviewer examine or interact with?

The reviewer should specifically interact with the design document and the UML diagram.

2) What questions do you want in this review to help answer?

Does the UML design effectively demonstrate how the proposed solution will address the problem outlined in the design document?

---

3) What should the reviewer focus on?

The reviewer should main center their focus on the UML document.

4) What should the reviewer *not* evaluate during this review?

The reviewer should not review the code we've written up to this point as the main goal of this sprint was to flesh out previous ideas and sponsor goals given in the previous sprint.

Examples:

- Do not evaluate visual polish or mobile responsiveness (for a usability test evaluating navigation and task completion)
  - Do not evaluate the database layer or unfinished API integration (for a code review focusing on the authentication module)
- 

5) What should the reviewer use to judge the work?

They should use the goals and layout in the design document and how well/clear the design for the project was conveyed through the UML diagram.

---

---

## Reviewer Section

1) Briefly describe what you actually did during the review.

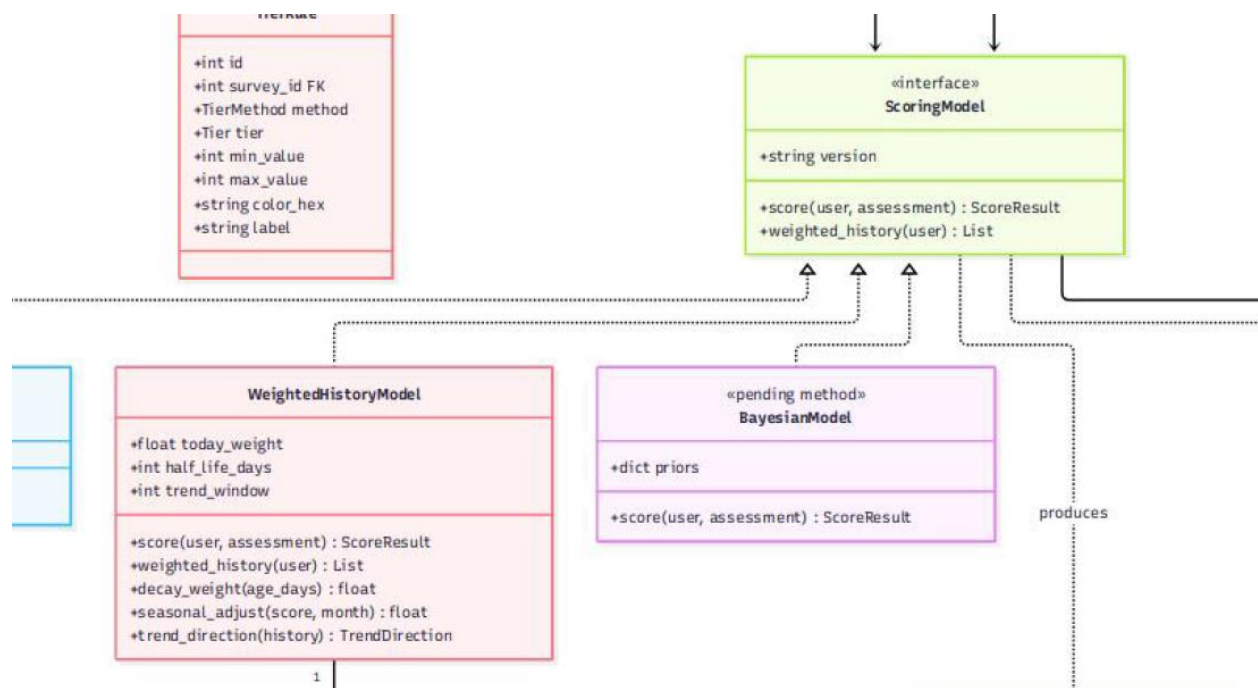
Examples include executing defined test cases, inspecting particular code paths, observing a user completing defined tasks, or systematically examining a design against specified criteria.

I reviewed the design documents and UML diagrams submitted by the group working on the Farmer Wellness app. I checked how they designed their system structure, how data moves through the app, and whether their design matches the sponsor's project requirements.

2) What portions of the artifact did you evaluate?

For this review, I evaluated the UML class diagram to see how key domain entities like User, Assessment, Question, Response, Progress Service, and Resource interact with one another. I also examined the sequence diagram to trace the step-by-step workflow of a farmer completing a survey and receiving results. Finally, I reviewed the design document to assess their feature user stories, scoring rules, open questions, and security specifications such as HIPAA compliance and data encryption at rest.

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.



## Findings

### Strengths:

- It was very organized and they did a really nice job keeping things readable. The layout clearly separates user accounts, survey questions, scoring logic, and local resources so nothing feels jumbled together.
- Also I liked that for scoring they used ScoringResult and WeightedHistoryModel that'll make it way easier for them to tweak or swap out how quizzes are calculated later without messing up the rest of their code.

### Issues / Defects:

| Description of Issue                | Severity (Low/Med/High) | Evidence / Example                                                                                                                                 | Suggestions                                                                             |
|-------------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Small mismatch between diagrams     | Low                     | The sequence diagram lists a generic "Scoring module", but the class diagram uses ProgressService and ScoringResult.                               | easy fix, but updating the sequence diagram so the names match would keep things clear. |
| Missing variables for tier cutoffs  | Low                     | Since the cutoff scores for Tiers 1, 2, and 3 are still TBD by the sponsor, adding placeholder variables would make it easier to plug in num later | Add placeholders                                                                        |
| Security requirements are drawn out | Medium                  | Audit logging and encrypted fields aren't shown in uml diagram                                                                                     | This becomes more of a priority in the later stages of the proj                         |

---

## Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.

Nishanth provided some valuable insights regarding the UML diagram, and I will use these observations to further improve the work done in Sprint 2.